BreachMirror-AI: One-Page Technical Documentation

A security policy assistant built on RAG and a LangGraph agentic workflow

Overview. BreachMirror-AI answers employee security questions from internal policy documents instead of relying on an LLM's memory. It delivers policy-based answers with citations, incident response guidance, and guardrails for sensitive information.

What was implemented

Component	What it does	Why
Policy knowledge base	`extract_handbook_documents()` reads `src/data/handbook-content.ts` and turns it into searchable documents (IT Security Policy, Incident Response Plan, Data Retention & Disposal).	Answers come from real company policy, not guesses.
Retrieval engine	`LocalPolicyRetriever` uses BM25Okapi to search policies before any response is generated.	Retrieval before generation reduces hallucinations.
Query expansion	Synonym mapping: wfh → remote work, vacation → annual leave, breach → security incident (also hack → security breach).	Employees rarely use the exact policy wording.
Guardrails	`redact_sensitive_info()` uses regex to redact API keys, passwords, secrets, bearer tokens and private keys. Example: `api_key=abc123secret` becomes [REDACTED SENSITIVE DETAIL].	Prevents accidental credential exposure.

Agentic workflow (LangGraph StateGraph)

Retrieve → **Draft** → **Evaluate** → **Finalize**

- **Retrieve** (`retrieve_documents()`): searches policies and passes the relevant ones downstream. Example: "remote work policy" returns the IT Security Policy.
- **Draft** (`draft_response()`): writes an answer from the retrieved evidence.
- **Evaluate** (`evaluate_response()`): scores 1 if a citation exists, otherwise 0. Answers without supporting evidence are rejected.
- **Retry**: a failed evaluation sends the flow back to Draft, with a maximum of 2 retries to prevent infinite loops.
- **Finalize** (`finalize_response()`): applies redaction before the answer reaches the user.

Compared with a single LLM call (Question → Answer), the system reasons, retrieves, drafts, reviews and finalizes, making several autonomous decisions before it responds.

Architecture

```
User question
   -> Local Policy Retriever (BM25) -> relevant policy
   -> LangGraph agent: Retrieve -> Draft -> Evaluate (retry if needed) -> Finalize (redaction)
   -> User response with citation
```

Testing performed

Query	Result	Citation
remote work	Remote work permitted up to 2 days weekly	IT Security Policy
phishing email	Report within 1 hour	Incident Response Plan
`api_key='secret'`	Guardrail check: confirms redaction runs before responses reach users	n/a

Where agentic AI is used

Component	Concept	Component	Concept
Document retrieval	Knowledge gathering	Retry mechanism	Autonomous decision making
LangGraph workflow	Agent orchestration	Finalization node	Safety enforcement
Evaluation node	Self-assessment	Citation validation	Grounded reasoning

Outcome

The app has grown from a simple policy lookup tool into an Agentic RAG system that retrieves policy knowledge before answering, produces evidence-based answers with citations, validates quality, self-corrects through retries, and applies security guardrails, all through a multi-step LangGraph workflow rather than one LLM call.